```
sudo pip uninstall fabric paramiko && sudo pip install
paramiko==<version shown in the error> && sudo pip install
fabric
```

Now, it's time to dirty our hands with Fabric, since it has been successfully installed, and verify it by running *fab -h* on the command line console, which produces some usage output. Fabric requires a file named *fabfile.py*, by default, to read the job definitions and their settings to trigger them for remote execution. Fabric jobs are Python sub-routines, so they could be as simple as running a simple command or could be complicated provisioning workflows. Shown below is a code snippet, which is a basic template of *fabfile*. It consists of a routine to dump *disk_usage*, information on processing cores, total runtime memory available and the version of Python installed:

```
from fabric.api import env, run, sudo, get, put

env.warn_only = True
env.skip_bad_hosts = True
env.colorize_errors = True
env.connection_attempts = 3

def dump():
  try:
     run('df -kh')
     run('grep -i processor /proc/cpuinfo')
     run('grep -i memtotal /proc/meminfo')
     run('python --version')

  except Exception, e:
     print(" Error: %s" %e)
```

You could view the job(s) to be remotely executed in *fabfile* by issuing the *fab -l* command and initiate those jobs on a number of remote servers using the command: *fab <Job> -H <server1>,<server2>,...,<serverN> -u <common user> -p <common password>*. You could also use a key pair instead of the password, using the option *-i* and run the *fabfile* routine(s) in parallel, with the option *-P* (the parallel execution is broken on Windows due to multi-processing and issues with Pickle modules there). In fact, the *fab* command provides a lot of options to run your job orchestration in a number of ways, and running the command *fab -h* gives users the help regarding all the options supported.

If it seems too much typing on the console and your options are fixed, then you could mention everything in the *fabfile*. You could also group different servers based upon the functionalities they offer, and then run similar or different jobs on different groups of servers. The *fabfile* shown below is more advanced, and all the options are set within it (note the use of *put/sudo/get* in various jobs, which is an example of lightweight non-centralised provisioning, using Fabric):

```
<start of code>

from fabric.api import env, run, sudo, get, put

env.user = "<sudo USER>"
env.key_filename = "<KEY PAIR>"
env.warn_only = True
env.skip_bad_hosts = True
env.colorize_errors = True
env.connection_attempts = 3

env.roledefs = {
                "WEB": [
                        "<webserverVM1>",
                        ...,
                        "<webserverVMn>",
                       ],
               "DATABASE": [
                            "<dbserverVM1>",
                            ...,
                            "<dbserverVMn>",
                           ],
              ...,
             }

def provWebServer():
  try:
    put("<webserver provisioning script>", "/tmp")
    sudo("cd /tmp && bash <webserver provisioning script>")
    get("/tmp/<webserver provisioning log>")
  except Exception, e:
    print(" Error: %s" %e)

def provDBServer():
  try:
    put("<dbserver provisioning script>", "/tmp")
    sudo("cd /tmp && bash <dbserver provisioning script>")
    get("/tmp/<dbserver provisioning log>")
  except Exception, e:
    print(" Error: %s" %e)

<end of code>
```

Now just run the command *fab provWebServer -R "WEB"* to provision all your servers that are required to act as Web servers and the command *fab provDBServer -R "DATABASE"* to provision database servers. Keeping all your provisioning logic in separate scripts ensures the provisioning and orchestration logic are separate, so you can change your provisioning needs just by changing the provisioning script. You could also create Fabric jobs that take parameters at runtime by issuing the *fab <JOB>: Param1,Param2,..* command format, by creating Python sub-routines taking various arguments.